GRUPO 2 · REQUERIMIENTOS FUNCIONALES

Transferencias entre Cuentas

Módulo: Transferencias internas (mismo banco)

Curso de Automatización de Pruebas de Software
Universidad Nacional de Asunción — CIT

Profesor: Steven Gracia

Campo	Valor
Documento	Requerimientos funcionales — Grupo 2
Versión	1.0
Fecha	22 de agosto de 2026
Sistemas alcanzados	aiquaa-sandbox-api (REST) y aiquaa-sandbox-web (front)
Fuente de la lógica	Código fuente de ambos repositorios y scripts SQL del schema qa_training
Uso previsto	Base para escribir escenarios BDD/Gherkin y casos de prueba automatizados

> Documento derivado del código fuente de aiquaa-sandbox-api y aiquaa-sandbox-web. Describe el comportamiento realmente implementado en el sandbox de práctica, no un sistema bancario productivo. El anexo normativo es material de referencia orientativo.

Contenido

1. Alcance y actores
2. Precondiciones y acceso
3. Modelo de datos del módulo
4. Requerimientos funcionales
5. Reglas transversales de la API
6. Flujo en la aplicación web
7. Criterios de aceptación
8. Anexo normativo (BCP / SEDECO) y brechas

1. Alcance y actores

Cubre el CRUD completo de cuentas y de transferencias internas entre dos cuentas del sandbox. La transferencia se registra como una instrucción en estado `pendiente`: queda asentada con su importe, sus cuentas y su descripción, y puede consultarse, reemplazarse o darse de baja por su identificador.

Fuera de alcance: Liquidación de la transferencia: el sandbox no descuenta ni acredita saldos, no valida fondos disponibles ni el estado de las cuentas, y no expone endpoints de confirmación, rechazo o reversa. Tampoco hay transferencias a otras entidades ni conversión de moneda.

Actores

Actor	Descripción
Alumno / QA automatizador	Consume la API con su propia API key y automatiza escenarios BDD sobre el módulo de su grupo.
Usuario de negocio (fila de `usuarios`)	Identidad de dominio sobre la que operan los endpoints: es dueño de cuentas, facturas, tarjetas, órdenes, reservas, notificaciones y roles.
API REST (aiquaa-sandbox-api)	Ejecuta la lógica de negocio con SQL fijo bajo el rol de base de datos `qa_api` (SELECT + INSERT + UPDATE, sin DELETE).
Aplicación web (aiquaa-sandbox-web)	Interfaz que consume la API a través del proxy `/api/proxy/{path}`; nunca llama al backend directamente desde el navegador.

Endpoints del módulo

Método y ruta	Requerimiento	Descripción
GET /api/v1/cuentas	RF-G2-01	Lista cuentas activas, opcionalmente filtradas por titular.
POST /api/v1/cuentas	RF-G2-05	Crea una cuenta para un titular (saldo inicial 0).
GET /api/v1/cuentas/{id}	RF-G2-02	Devuelve el detalle de una cuenta.
PUT /api/v1/cuentas/{id}	RF-G2-06	Reemplaza tipo y moneda de una cuenta.
DELETE /api/v1/cuentas/{id}	RF-G2-07	Da de baja (soft-delete) una cuenta.
GET /api/v1/transferencias	RF-G2-08	Lista transferencias activas, por cuenta de origen o destino.
POST /api/v1/transferencias	RF-G2-03	Registra una transferencia entre dos cuentas.
GET /api/v1/transferencias/{id}	RF-G2-04	Devuelve el detalle de una transferencia.
PUT /api/v1/transferencias/{id}	RF-G2-09	Reemplaza cuentas, importe y descripción de una transferencia.
DELETE /api/v1/transferencias/{id}	RF-G2-10	Da de baja (soft-delete) una transferencia.

2. Precondiciones y acceso

- Toda request a la API debe enviar el header `x-api-key` con una clave existente y con `active = true` en la tabla `public.api_keys`; sin header o con clave inválida/inactiva la respuesta es `401 UNAUTHORIZED` con el mensaje genérico "Invalid or inactive API key.".

- Cada API key admite un máximo de **200 requests por minuto** (ventana deslizante). Superado el límite la API responde `429 RATE_LIMITED` con los headers `Retry-After`, `X-RateLimit-Limit`, `X-RateLimit-Remaining` y `X-RateLimit-Reset`.
- Los datos de práctica viven en el schema aislado `qa_training` y se cargan con `scripts/seed-data.sql`; la data es determinística, por lo que un escenario puede apoyarse en los registros sembrados.
- La API no expone borrado físico: el rol `qa_api` no tiene permiso `DELETE` en Postgres, y todo endpoint `DELETE` de la API hace baja lógica (`activo = false`), devolviendo `204 No Content` sin cuerpo.
- Casi todas las tablas de negocio (`sesiones`, `transferencias`, `facturas`, `tarjetas`, `notificaciones`, `ordenes`, `reservas`, `movimientos`, `roles`, además de `usuarios` y `cuentas` que ya la tenían) tienen columna `activo`/`activa`; todo `GET`, `PUT` y `DELETE` de esa tabla filtra u opera solo sobre filas con `activo = true` — una fila dada de baja deja de ser visible y de poder reemplazarse, pero no se borra.
- En la aplicación web el acceso tiene dos capas: capa 1 la API key (pantalla `/login`, se valida contra `GET /api/v1/roles` antes de guardarse) y capa 2 el usuario de negocio (pantalla `/auth/login`). Con `NEXT_PUBLIC_DEMO_MODE=true` la capa 1 se omite y el proxy inyecta una key demo del servidor.
- Las rutas `/usuarios/*` de la web son la excepción al guard de capa 2: se pueden usar solo con la API key, porque son el punto de partida para obtener un `usuarioId`.
- Para los escenarios de transferencia se necesitan al menos dos cuentas distintas ya existentes; los datos sembrados incluyen varias cuentas por titular, en guaraníes y en dólares.
- Las cuentas se identifican por su número interno (`id`), no por el número de cuenta comercial.

3. Modelo de datos del módulo

cuentas

Cuentas de los titulares. Ningún endpoint modifica el saldo; `numero_cuenta` lo genera la API al crear la cuenta.

Columna	Tipo / restricción
`id`	`bigserial`, clave primaria
`usuario_id`	`bigint`, obligatorio, referencia a `usuarios(id)`
`numero_cuenta`	`text`, obligatorio y único; lo genera `POST /cuentas` al azar (10 dígitos)
`tipo_cuenta`	`text`, `ahorro` o `corriente` (por defecto `ahorro`)
`moneda`	`text`, `PYG` o `USD` (por defecto `PYG`)
`saldo`	`numeric(14,2)`, obligatorio, por defecto 0; ningún endpoint lo modifica
`activa`	`boolean`, obligatorio, por defecto `true`; la usan `GET/PUT/DELETE` para filtrar/operar
`created_at`	`timestamptz`, por defecto `now()`

transferencias

Instrucciones de transferencia registradas. Es la tabla que escribe el módulo.

Columna	Tipo / restricción
`id`	`bigserial`, clave primaria
`cuenta_origen_id`	`bigint`, obligatorio, referencia a `cuentas(id)`
`cuenta_destino_id`	`bigint`, obligatorio, referencia a `cuentas(id)`; la base impide que sea igual a la cuenta de origen
`monto`	`numeric(14,2)`, obligatorio, la base exige que sea mayor que cero
`descripcion`	`text`, opcional
`estado`	`text`, uno de `pendiente` / `completada` / `rechazada`; por defecto `pendiente`
`created_at`	`timestamptz`, por defecto `now()`
`activo`	`boolean`, obligatorio, por defecto `true`; lo usan `GET/PUT/DELETE` para filtrar/operar

> El saldo de las cuentas es informativo para este módulo: registrar una transferencia por un importe mayor al saldo disponible es aceptado por el sistema. Los escenarios no deben esperar que el saldo cambie después de transferir.

4. Requerimientos funcionales

RF-G2-01 — Listar cuentas

`GET /api/v1/cuentas`

Devuelve las cuentas del sandbox, con la posibilidad de acotar el resultado a un titular determinado. Es el paso previo para elegir cuenta de origen y de destino.

Entradas y validaciones

- `usuarioId` (opcional, por query): número entero positivo. Se envía como texto en la URL y el sistema lo convierte.

Reglas de negocio

- Devuelve únicamente cuentas con `activa = true`; una dada de baja con RF-G2-07 deja de aparecer.
- Con `usuarioId`, devuelve todas las cuentas activas de ese titular ordenadas por identificador ascendente, sin límite de cantidad.
- Sin `usuarioId`, devuelve las primeras 100 cuentas activas del sandbox ordenadas por identificador ascendente.
- Un titular sin cuentas, o un `usuarioId` inexistente, devuelve una lista vacía y no un error.
- No hay paginación: el resultado sin filtro está acotado por el tope fijo de 100 registros.

Respuesta esperada

- `200 OK` con `data` como arreglo de cuentas; cada elemento incluye `id`, `usuario_id`, `numero_cuenta`, `tipo_cuenta`, `moneda`, `saldo`, `activa` y `created_at`.
- El campo `saldo` viaja como texto con dos decimales, por ser un valor numérico exacto de la base.

Errores

Código	HTTP	Cuándo ocurre
`VALIDATION_ERROR`	400	`usuarioId` presente pero no numérico, cero o negativo.
`UNAUTHORIZED`	401	Falta la API key o es inválida.

Fuente: `app/api/v1/cuentas/route.ts`

RF-G2-02 — Consultar una cuenta

`GET /api/v1/cuentas/{id}`

Devuelve el detalle de una cuenta puntual, identificada por su número interno.

Entradas y validaciones

- `id` (obligatorio, en la ruta): número entero positivo.

Reglas de negocio

- Devuelve la cuenta cuyo identificador coincide exactamente.
- Si no existe o ya está dada de baja (`activa = false`), responde 404 con el mensaje "Cuenta no encontrada.".
- El identificador de la ruta tiene prioridad sobre cualquier `id` enviado por query o en el cuerpo.

Respuesta esperada

- `200 OK` con `data` conteniendo la cuenta completa.

Errores

Código	HTTP	Cuándo ocurre
`NOT_FOUND`	404	No existe una cuenta con ese identificador.
`VALIDATION_ERROR`	400	El identificador no es un entero positivo.

Fuente: `app/api/v1/cuentas/[id]/route.ts`

RF-G2-03 — Registrar una transferencia

`POST /api/v1/transferencias`

Registra una instrucción de transferencia entre dos cuentas del sandbox por un importe determinado. La instrucción nace en estado `pendiente` y no altera los saldos.

Entradas y validaciones

- `cuentaOrigenId` (obligatorio): número entero positivo.
- `cuentaDestinoId` (obligatorio): número entero positivo.
- `monto` (obligatorio): número mayor que cero. Se acepta como texto numérico y el sistema lo convierte.
- `descripcion` (opcional): texto libre; si se omite se guarda vacío.

Reglas de negocio

- La transferencia se registra siempre con `estado = 'pendiente'`; el cliente no puede elegir el estado.
- La cuenta de destino debe ser distinta de la de origen: la base rechaza el registro si coinciden.
- El importe debe ser mayor que cero, validado tanto en la entrada como en la base de datos.
- Ambas cuentas deben existir; si alguna no existe, la operación falla como error de ejecución por violación de clave foránea.
- **No se valida saldo suficiente, ni que las cuentas estén activas, ni que compartan moneda, ni que pertenezcan al mismo titular.** Una transferencia por un importe superior al saldo, entre monedas distintas o desde una cuenta inactiva es aceptada.
- Los saldos de origen y destino quedan intactos: la operación es puramente registral.

Respuesta esperada

- `201 Created` con `data` conteniendo la transferencia creada: `id`, cuentas, `monto`, `descripcion`, `estado` (`pendiente`) y `created_at`.

Errores

Código	HTTP	Cuándo ocurre
`VALIDATION_ERROR`	400	Falta un campo obligatorio, el importe no es positivo, un identificador no es entero positivo, la cuenta de origen o de destino no existe, o ambas son la misma cuenta.

Fuente: `app/api/v1/transferencias/route.ts` y las restricciones de `scripts/setup-db.sql`

RF-G2-04 — Consultar una transferencia

`GET /api/v1/transferencias/{id}`

Devuelve el detalle de una transferencia registrada, para verificar su importe, sus cuentas y su estado.

Entradas y validaciones

- `id` (obligatorio, en la ruta): número entero positivo.

Reglas de negocio

- Devuelve la transferencia cuyo identificador coincide exactamente.
- Si no existe o ya está dada de baja (`activo = false`), responde 404 con el mensaje "Transferencia no encontrada.".
- El estado devuelto es siempre `pendiente` para las transferencias creadas por la API; los datos sembrados incluyen además ejemplos en `completada` y `rechazada`.

Respuesta esperada

- `200 OK` con `data` conteniendo la transferencia completa.

Errores

Código	HTTP	Cuándo ocurre
`NOT_FOUND`	404	No existe una transferencia con ese identificador.
`VALIDATION_ERROR`	400	El identificador no es un entero positivo.

Fuente: `app/api/v1/transferencias/[id]/route.ts`

RF-G2-05 — Crear una cuenta

`POST /api/v1/cuentas`

Crea una cuenta nueva para un titular, con saldo inicial cero y número de cuenta generado por el sistema.

Entradas y validaciones

- `usuarioId` (obligatorio): número entero positivo del titular.
- `tipoCuenta` (obligatorio): `ahorro` o `corriente`.
- `moneda` (obligatorio): `PYG` o `USD`.

Reglas de negocio

- El `numeroCuenta` no lo envía el cliente: lo genera el servidor al azar (10 dígitos) y debe ser único en todo el sandbox.
- La cuenta se crea siempre con `saldo = 0` y `activa = true`.
- El titular debe existir: un `usuarioId` inexistente falla como error de ejecución por violación de clave foránea.
- En la práctica improbable de que el número generado colisione con uno existente, la base rechaza el INSERT por violación de unicidad (`409 CONFLICT`); no hay reintento automático.

Respuesta esperada

- `201 Created` con `data` conteniendo la cuenta creada.

Errores

Código	HTTP	Cuándo ocurre
`VALIDATION_ERROR`	400	Falta un campo obligatorio, `tipoCuenta`/`moneda` tienen un valor fuera de la lista permitida, o el `usuarioId` indicado no existe.
`CONFLICT`	409	El número de cuenta generado al azar ya existía (caso extremadamente improbable).

Fuente: `app/api/v1/cuentas/route.ts`

RF-G2-06 — Reemplazar una cuenta

`PUT /api/v1/cuentas/{id}`

Reemplaza el tipo y la moneda de una cuenta existente. El número de cuenta y el saldo quedan fuera de este reemplazo.

Entradas y validaciones

- `id` (obligatorio, en la ruta): número entero positivo.
- `tipoCuenta` (obligatorio): `ahorro` o `corriente`.
- `moneda` (obligatorio): `PYG` o `USD`.

Reglas de negocio

- Solo puede reemplazar una cuenta con `activa = true`; una dada de baja responde 404.
- `numeroCuenta` y `saldo` no forman parte del cuerpo: no pueden reemplazarse por esta vía.

Respuesta esperada

- `200 OK` con `data` conteniendo la cuenta ya actualizada.

Errores

Código	HTTP	Cuándo ocurre
`NOT_FOUND`	404	No existe una cuenta vigente con ese identificador.
`VALIDATION_ERROR`	400	Falta un campo obligatorio o tiene un valor fuera de la lista permitida.

Fuente: `app/api/v1/cuentas/[id]/route.ts`

RF-G2-07 — Dar de baja una cuenta

`DELETE /api/v1/cuentas/{id}`

Da de baja lógica una cuenta, quitándola de los listados y consultas por id.

Entradas y validaciones

- `id` (obligatorio, en la ruta): número entero positivo.

Reglas de negocio

- Marca `activa = false`; la fila permanece en la base, nunca se borra físicamente.
- No verifica si la cuenta tiene transferencias activas asociadas: una cuenta con transferencias vigentes puede darse de baja igual, y esas transferencias siguen existiendo y siendo consultables por su propio id.

Respuesta esperada

- `204 No Content`, sin cuerpo.

Errores

Código	HTTP	Cuándo ocurre
`NOT_FOUND`	404	No existe una cuenta vigente con ese identificador.
`VALIDATION_ERROR`	400	El identificador no es un entero positivo.

Fuente: `app/api/v1/cuentas/[id]/route.ts`

RF-G2-08 — Listar transferencias

`GET /api/v1/transferencias`

Devuelve las transferencias del sandbox, con la posibilidad de filtrar por cuenta de origen o de destino.

Entradas y validaciones

- `cuentaOrigenId` (opcional, por query): número entero positivo.
- `cuentaDestinoId` (opcional, por query): número entero positivo.

Reglas de negocio

- Devuelve únicamente transferencias con `activo = true`.
- Los dos filtros no se combinan entre sí: si se envía `cuentaOrigenId`, se ignora `cuentaDestinoId` (y viceversa); solo el primero que la ruta evalúa (`cuentaOrigenId`) tiene efecto cuando ambos están presentes.
- Sin ningún filtro, devuelve las primeras 100 transferencias del sandbox ordenadas por identificador ascendente.

Respuesta esperada

- `200 OK` con `data` como arreglo de transferencias.

Errores

Código	HTTP	Cuándo ocurre
`VALIDATION_ERROR`	400	Alguno de los filtros está presente pero no es un entero positivo.
`UNAUTHORIZED`	401	Falta la API key o es inválida.

Fuente: `app/api/v1/transferencias/route.ts`

RF-G2-09 — Reemplazar una transferencia

`PUT /api/v1/transferencias/{id}`

Reemplaza las cuentas, el importe y la descripción de una transferencia existente. El estado queda fuera de este reemplazo.

Entradas y validaciones

- `id` (obligatorio, en la ruta): número entero positivo.
- `cuentaOrigenId` (obligatorio): número entero positivo.
- `cuentaDestinoId` (obligatorio): número entero positivo.
- `monto` (obligatorio): número mayor que cero.
- `descripcion` (opcional): texto libre.

Reglas de negocio

- Solo puede reemplazar una transferencia con `activo = true`; una dada de baja responde 404.
- `estado` no forma parte del cuerpo: no hay forma de mover una transferencia de `pendiente` a otro estado por esta vía (no existe ningún endpoint que lo haga).
- Las mismas restricciones de la creación aplican aquí: cuentas existentes, destino distinto del origen, importe positivo.

Respuesta esperada

- `200 OK` con `data` conteniendo la transferencia ya actualizada.

Errores

Código	HTTP	Cuándo ocurre
`NOT_FOUND`	404	No existe una transferencia vigente con ese identificador.
`VALIDATION_ERROR`	400	Falta un campo obligatorio, el importe no es positivo, alguna cuenta no existe, o ambas cuentas son la misma.

Fuente: `app/api/v1/transferencias/[id]/route.ts`

RF-G2-10 — Dar de baja una transferencia `DELETE /api/v1/transferencias/{id}`

Da de baja lógica una transferencia, quitándola de los listados y consultas por id.

Entradas y validaciones

- `id` (obligatorio, en la ruta): número entero positivo.

Reglas de negocio

- Marca `activo = false`; la fila permanece en la base, nunca se borra físicamente.
- No cambia el `estado` de la transferencia (sigue en `pendiente` u otro valor sembrado): dar de baja y cambiar de estado son cosas distintas.

Respuesta esperada

- `204 No Content`, sin cuerpo.

Errores

Código	HTTP	Cuándo ocurre
`NOT_FOUND`	404	No existe una transferencia vigente con ese identificador.
`VALIDATION_ERROR`	400	El identificador no es un entero positivo.

Fuente: `app/api/v1/transferencias/[id]/route.ts`

5. Reglas transversales de la API

Todas las rutas REST comparten el mismo pipeline: autenticación → control de caudal → validación de entrada → ejecución del SQL fijo → auditoría. Estas reglas aplican a cada requerimiento del documento y no se repiten en cada uno.

- **Envoltura de respuesta:** las respuestas exitosas devuelven `{ "data": ... }` y las fallidas `{ "error": { "code", "message", "details?" } }`.
- **Validación de entrada:** cada ruta define un esquema; los parámetros de query, el cuerpo JSON y los parámetros de ruta se combinan en un único objeto con precedencia query < body < path (un `{id}` de la URL siempre gana sobre un `id` del cuerpo).
- **Coerción de tipos:** los identificadores y montos que llegan por query o por path se convierten de texto a número automáticamente; los campos numéricos de un cuerpo JSON deben enviarse como número, no como texto, salvo que el requerimiento indique lo contrario.
- **Cuerpo JSON malformado:** un cuerpo que no parsea como JSON devuelve `400 VALIDATION_ERROR` con el mensaje "Invalid JSON body.".
- **Errores de base de datos, mapeados por código SQLSTATE:** una violación de unicidad (`23505`, ej. un email o número de factura repetido) se traduce a `409 CONFLICT`; una violación de clave foránea o de restricción `CHECK` (`23503` / `23514`) se traduce a `400 VALIDATION_ERROR`; cualquier otro error de base de datos cae en `400 EXECUTION_ERROR`.
- `DELETE` **es siempre baja lógica:** actualiza `activo = false` (o `activa` en `cuentas`) y responde `204 No Content` **sin cuerpo** — no `200` con la fila afectada.
- **Verbo no soportado:** una ruta que no define el método HTTP usado responde `405 Method Not Allowed`; lo genera Next.js automáticamente, ninguna ruta lo arma a mano, y su cuerpo no sigue la

envoltura `{ error }` del resto de la API.

- **Auditoría:** toda request, exitosa o fallida, deja un registro en `public.sql_audit_log` con la clave usada, el método y la ruta, la entrada validada, el resultado y la IP de origen. Un fallo de auditoría nunca convierte una respuesta exitosa en error.
- **Idempotencia de las transiciones de estado:** los endpoints que fijan un estado (`bloquear`, `activar`, `confirmar`, `cancelar`, `leer`) escriben el valor destino sin verificar el estado previo, por lo que repetir la llamada devuelve el mismo resultado.
- **Las transiciones de estado anteriores al CRUD no miran `activo`:** `PATCH .../bloquear`, `.../activar`, `.../leer`, `.../confirmar`, `.../cancelar`, `.../kyc` y `POST .../pagar` no filtran por `activo = true` (ninguno de esos archivos se tocó al agregar el CRUD). Un recurso dado de baja con el nuevo `DELETE` deja de aparecer en `GET`, pero **sigue pudiendo bloquearse, confirmarse, leerse o pagarse** a través de su acción dedicada — una inconsistencia real del sandbox, no un comportamiento a asumir como intencional.

Códigos de éxito

Status	Cuándo ocurre
`200 OK`	GET/PUT/PATCH exitoso; también un POST de upsert que reactivó una fila existente en vez de crearla.
`201 Created`	POST que crea una fila nueva.
`204 No Content`	DELETE exitoso (baja lógica) — la respuesta no lleva cuerpo.

Códigos de error

Código	HTTP	Significado
`UNAUTHORIZED`	401	Falta el header `x-api-key`, o la clave no existe o está inactiva.
`RATE_LIMITED`	429	Se superaron las 200 requests por minuto de esa API key.
`VALIDATION_ERROR`	400	La entrada no cumple el esquema de la ruta, el JSON es inválido, o Postgres rechazó una clave foránea/CHECK.
`EXECUTION_ERROR`	400	La consulta falló en la base de datos por un motivo no cubierto por los códigos anteriores.
`NOT_FOUND`	404	El recurso indicado no existe, ya está dado de baja (`activo=false`), o no está en un estado que admita la operación.
`CONFLICT`	409	Violación de restricción `UNIQUE` en Postgres (ej. email, número de factura o nombre de rol duplicado).
`INTERNAL_ERROR`	500	Fallo inesperado del servidor (por ejemplo, no se pudo verificar la API key).

6. Flujo en la aplicación web

La aplicación web consume exactamente los mismos endpoints a través del proxy `/api/proxy/{path}`, que agrega la API key del lado del servidor y reenvía los headers de límite de caudal. El menú de navegación

muestra únicamente los módulos del grupo del alumno cuando su email figura en el roster del curso; si no figura, muestra los diez módulos. Los errores de la API se muestran en pantalla con el mensaje que devolvió el backend, y un `401` cierra la sesión local. **La interfaz web todavía no tiene pantallas para `PUT` / `DELETE` ni para los recursos nuevos (`sesiones`, `movimientos`)**: solo cubre los flujos de creación y consulta que ya existían; esos requerimientos se prueban llamando la API directamente (Postman, `curl`, o el cliente HTTP del framework de automatización), no navegando la web.

Pantalla	Qué permite hacer
`/cuentas` — Listado de cuentas	Tabla con número, tipo, moneda, saldo y estado de cada cuenta, con un filtro por titular. Ejecuta RF-G2-01.
`/cuentas/{id}` — Detalle de cuenta	Muestra los datos de una cuenta puntual. Ejecuta RF-G2-02.
`/transferencias` — Nueva transferencia	La raíz del módulo es directamente el formulario de creación: cuenta de origen, cuenta de destino, importe y descripción opcional. Ejecuta RF-G2-03 y, al confirmarse, navega al detalle de la transferencia creada.
`/transferencias/{id}` — Detalle de transferencia	Muestra el comprobante de la transferencia registrada. Ejecuta RF-G2-04.

- El formulario web pide los identificadores numéricos de las cuentas, no los números de cuenta comerciales.
- El campo de importe del formulario exige un valor mínimo de 0,01, por lo que el rechazo de un importe cero ocurre en el navegador antes de llegar a la API.
- Si la API rechaza la transferencia, el mensaje de error del backend se muestra dentro del formulario y no se navega al detalle.

7. Criterios de aceptación

Criterios de aceptación redactados en prosa, uno o más por requerimiento, cubriendo el camino feliz y los caminos negativos. Son la base para derivar escenarios BDD.

RF-G2-01 — Listar cuentas

- Al listar cuentas sin filtro, la respuesta es 200 y `data` es un arreglo con como máximo 100 elementos ordenados por `id` ascendente.
- Al listar cuentas filtrando por un titular con cuentas, todos los elementos devueltos tienen ese `usuario_id`.
- Al filtrar por un `usuarioId` que no tiene cuentas, la respuesta es 200 y `data` es un arreglo vacío.
- Al enviar `usuarioId=abc`, la respuesta es 400 `VALIDATION_ERROR`.

RF-G2-02 — Consultar una cuenta

- Dada una cuenta existente, al consultarla por su identificador la respuesta es 200 y `data.id` coincide con el solicitado.
- Al consultar un identificador inexistente, la respuesta es 404 `NOT_FOUND` con el mensaje "Cuenta no encontrada.".
- Al consultar con un identificador no numérico, la respuesta es 400 `VALIDATION_ERROR`.

RF-G2-03 — Registrar una transferencia

- Dadas dos cuentas distintas existentes, al registrar una transferencia válida la respuesta es 201 y `data.estado` es `pendiente`.
- Después de registrar la transferencia, el saldo de la cuenta de origen consultado con RF-G2-02 es el mismo que antes de la operación.
- Al registrar una transferencia con la misma cuenta como origen y destino, la respuesta es 400 `VALIDATION_ERROR` (viola la restricción `CHECK` de la tabla).
- Al registrar una transferencia con importe cero o negativo, la respuesta es 400 `VALIDATION_ERROR`.
- Al registrar una transferencia contra una cuenta de destino inexistente, la respuesta es 400 `VALIDATION_ERROR` (viola la clave foránea) y ninguna transferencia queda registrada.
- Al registrar una transferencia por un importe mayor al saldo de la cuenta de origen, la respuesta es 201: el sandbox no controla fondos.
- La transferencia recién creada puede consultarse con RF-G2-04 usando el `id` devuelto.

RF-G2-04 — Consultar una transferencia

- Dada una transferencia recién registrada, al consultarla la respuesta es 200 y su `monto` y cuentas coinciden con los enviados al crearla.
- Al consultar una transferencia inexistente, la respuesta es 404 `NOT_FOUND` con el mensaje "Transferencia no encontrada.".
- Consultar dos veces la misma transferencia devuelve exactamente el mismo estado: no hay procesamiento diferido que lo cambie.

RF-G2-05 — Crear una cuenta

- Al crear una cuenta para un titular existente, la respuesta es 201, `data.saldo` es `0` y `data.activa` es verdadero.
- La cuenta creada aparece de inmediato en el listado de RF-G2-01 filtrando por ese titular.
- Al crear con `moneda=EUR`, la respuesta es 400 `VALIDATION_ERROR`.
- Al crear para un `usuarioId` inexistente, la respuesta es 400 `VALIDATION_ERROR`.

RF-G2-06 — Reemplazar una cuenta

- Dada una cuenta existente, al cambiar su `tipoCuenta` la respuesta es 200 y el cambio se refleja al volver a consultarla.
- El `numeroCuenta` y el `saldo` de la cuenta no cambian tras el reemplazo.
- Al reemplazar una cuenta inexistente o dada de baja, la respuesta es 404 `NOT_FOUND`.

RF-G2-07 — Dar de baja una cuenta

- Dada una cuenta vigente, al darla de baja la respuesta es 204 sin cuerpo.
- Tras la baja, `GET /cuentas/{id}` sobre esa cuenta responde 404, y deja de aparecer en el listado de RF-G2-01.
- Dar de baja una cuenta con transferencias existentes no afecta la consulta de esas transferencias por su propio id.

RF-G2-08 — Listar transferencias

- Al listar filtrando por `cuentaOrigenId`, todos los elementos devueltos tienen esa cuenta como origen.
- Al listar filtrando por `cuentaDestinoId`, todos los elementos devueltos tienen esa cuenta como destino.
- Al enviar ambos filtros a la vez, el resultado corresponde solo a `cuentaOrigenId`, no a la intersección de ambos.
- Una transferencia dada de baja con RF-G2-10 deja de aparecer en este listado.

RF-G2-09 — Reemplazar una transferencia

- Dada una transferencia existente, al reemplazar su `monto` la respuesta es 200 y el nuevo importe se refleja al consultarla.
- El `estado` de la transferencia no cambia por este reemplazo.
- Al reemplazar con la misma cuenta como origen y destino, la respuesta es 400 `VALIDATION_ERROR`.
- Al reemplazar una transferencia inexistente o dada de baja, la respuesta es 404 `NOT_FOUND`.

RF-G2-10 — Dar de baja una transferencia

- Dada una transferencia vigente, al darla de baja la respuesta es 204 sin cuerpo.
- Tras la baja, `GET /transferencias/{id}` sobre esa transferencia responde 404, y deja de aparecer en RF-G2-08.

8. Anexo normativo (BCP / SEDECO) y brechas

> Este anexo es material de referencia orientativo para dar contexto de negocio a los escenarios de prueba. Las normas citadas del Banco Central del Paraguay (BCP) y de la Secretaría de Defensa del Consumidor y el Usuario (SEDECO) se mencionan de forma general y deben validarse con su texto vigente antes de usarse como criterio de cumplimiento. El sandbox es un entorno didáctico y no pretende cumplir ninguna de ellas.

Referencia normativa	Expectativa	Estado en el sandbox
Reglamentación del BCP sobre el Sistema de Pagos del Paraguay (SIPAP) y transferencias electrónicas	Toda orden de transferencia debe tener identificación unívoca, trazabilidad de origen y destino y un estado final de liquidación.	Parcial: la orden se identifica y se traza (incluida la auditoría de la request), pero nunca alcanza un estado final: no hay liquidación.
Marco del BCP sobre disponibilidad de fondos en cuentas de depósito	Una orden solo puede ejecutarse si la cuenta ordenante tiene fondos suficientes y se encuentra operativa.	No implementado: no se valida saldo, ni que la cuenta esté activa; el saldo nunca se debita.
Ley 1015/97 y debida diligencia de operaciones (SEPRELAD)	Registro de las operaciones con datos suficientes para su monitoreo y detección de operaciones inusuales.	Parcial: cada transferencia queda registrada con importe, cuentas y fecha, pero no hay control de umbrales ni alertas.
Ley 1334/98 de Defensa del Consumidor (SEDECO), deber de información	El usuario debe recibir constancia de la operación con importe, destino y estado.	Cubierto: la API devuelve el comprobante completo y la web lo muestra en la pantalla de detalle.
Reglas de conversión de moneda del BCP	Una transferencia entre cuentas de distinta moneda requiere un tipo de cambio explícito.	No implementado: se admite transferir entre cuentas en guaraníes y en dólares sin conversión alguna.

Brechas conocidas del sandbox

- El registro de la transferencia no modifica `cuentas.saldo`: no hay débito ni crédito.
- No existe validación de fondos suficientes ni endpoint que complete o rechace una transferencia pendiente.
- No se valida que las cuentas estén activas ni que compartan moneda; tampoco que pertenezcan a titulares distintos o al mismo titular.
- El estado `completada` y `rechazada` solo aparece en los datos sembrados: la API nunca los produce.
- No hay límites por operación ni por período, ni control de duplicados (dos transferencias idénticas seguidas son aceptadas).
- El listado de cuentas sin filtro está limitado a 100 registros sin paginación ni indicación de que hubo recorte.
- `GET /transferencias` no combina `cuentaOrigenId` y `cuentaDestinoId`: si se envían ambos, solo se aplica el primero.
- Dar de baja una cuenta no verifica ni bloquea sus transferencias asociadas; tampoco existe cascada de baja entre ambas tablas.
- `PUT /transferencias/{id}` no permite tocar el `estado`, y sigue sin existir ningún endpoint que mueva una transferencia de `pendiente` a `completada` o `rechazada`.